

URL shortener: architecture

Components, runtime and data views, deployment, event pipeline and the agentic SDLC orchestrator.

1. Purpose and scope

This document describes the structure of the system: its components, how they connect, how requests and data flow, the deployment options, and the architecture of the agentic SDLC orchestrator that delivered part of the service. Rationale for the choices is in the companion design document.

Part	Role
<code>urlshort/</code>	URL shortener service: HTTP API, business rules, storage, analytics, identity, observability
<code>urlshort/adapters/</code>	Optional infrastructure: PostgreSQL, Redis cache and limits, Redis Streams, workers
<code>sdlc/</code>	Governed agentic SDLC orchestrator: engine, policy, approvals, audit, 12 agents, CLI
<code>scenarios/</code>	Greenfield, brownfield and ambiguous runs of the orchestrator against the service
<code>deploy/</code> , <code>docker-compose.yml</code> , <code>Dockerfile</code>	Container image, local Postgres/Redis, Prometheus/Grafana configuration

2. Architectural style

- **Hexagonal (ports and adapters).** HTTP routers are adapters; ShortenerService holds the business rules and imports no web framework; storage is a Repository port with SQLite and PostgreSQL adapters.
- **Stateless API instances.** All state lives in the database (and optionally Redis), so instances scale horizontally behind a load balancer.
- **Opt-in infrastructure.** Without configuration the service runs on SQLite in one process. Postgres, Redis and the event pipeline are enabled by environment variables, with no code change.
- **Governance as an engine.** In the orchestrator, agents produce artifacts; a separate engine evaluates gates, policies and approvals. Agents never decide whether their own work passes.

3. Service component view

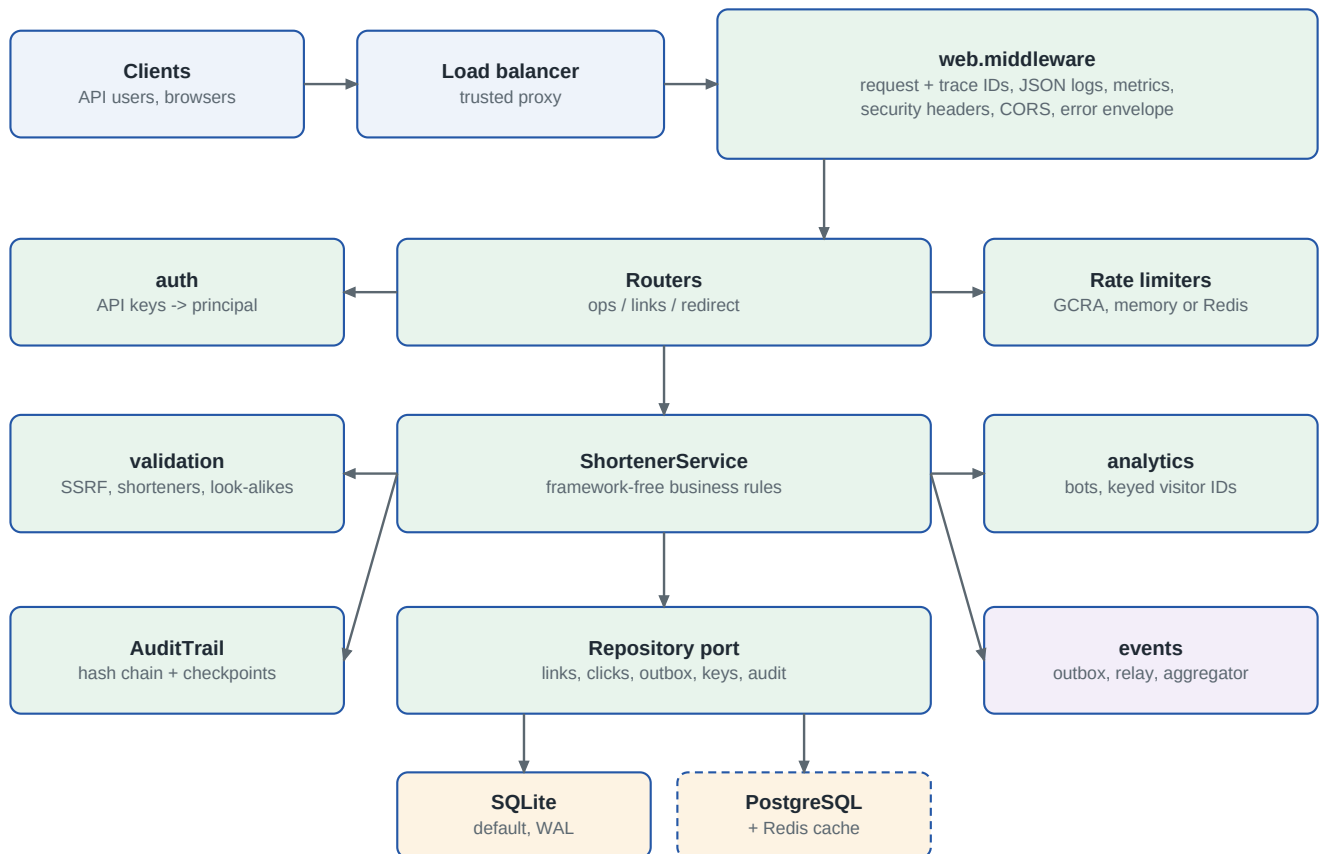


Figure 1. Service components. Dashed: optional PostgreSQL + Redis adapter.

Component	Responsibility
web.middleware	Request IDs, trace spans, JSON request logs, metrics, security headers, CORS, single error envelope
Routers	ops (/livez, /readyz, /metrics), links (/api/v1/links ...), redirect (/code), registered last
auth	Resolves X-API-Key / Bearer keys to a principal (owner or admin); invalid keys are 401
Rate limiters	GCRA per client for creates and redirects; in memory or shared in Redis
ShortenerService	Create (idempotent), resolve (expiry, caps, click recording), takedown, stats; no HTTP
validation	URL safety: schemes, SSRF, credentials, shorteners, look-alike hostnames; alias, TTL, cap rules
analytics	Bot and browser detection, referrer domains, daily keyed visitor IDs, summaries
AuditTrail	Hash-chained change log; signed checkpoints
events	Outbox relay and idempotent aggregator (events mode)
Repository	Port for links, clicks, outbox, API keys and audit; SQLite (default) and PostgreSQL adapters

4. Runtime view: the redirect path

Step	What happens	Notes
1	Middleware assigns a request ID and continues or starts a trace	traceparent honoured; IDs on logs and headers
2	Client IP resolved (trusted proxies only); redirect rate limit applied	429 with Retry-After when exceeded
3	Link looked up (Bloom filter, then cache, then database when Redis is on)	Unknown codes: 404 without a DB query
4	Inactive -> 404; expired -> 410	Takedowns apply on every instance at once
5a	Capped link: one conditional UPDATE checks and counts atomically	Cap reached: 410; storage error: 503 (fail closed)
5b	Uncapped link: click recorded (or written to the outbox)	Storage error: redirect continues (fail open)
6	307 to the target with Cache-Control: no-store and security headers	Metrics by route template; span closed

5. Data view

Table	Purpose	Key properties
links	One row per short link	code is the primary key; soft delete; stats token hash; max_clicks; owner
clicks	One row per redirect	Referrer domain only; browser family; bot flag; keyed visitor ID; unique event_id
click_outbox	Events waiting to be published	Written in the same transaction as the counter; published_at
api_keys	Owner and admin keys	SHA-256 of the secret only; role, owner, revocation time
audit_log	Hash-chained change history	Appended under a lock (SQLite: BEGIN IMMEDIATE; Postgres: advisory lock)

Schema version 5. Migrations are expand-only (additive, nullable): SQLite uses PRAGMA user_version; PostgreSQL uses reversible Alembic revisions named after the same versions, with a parity test.

6. Deployment view

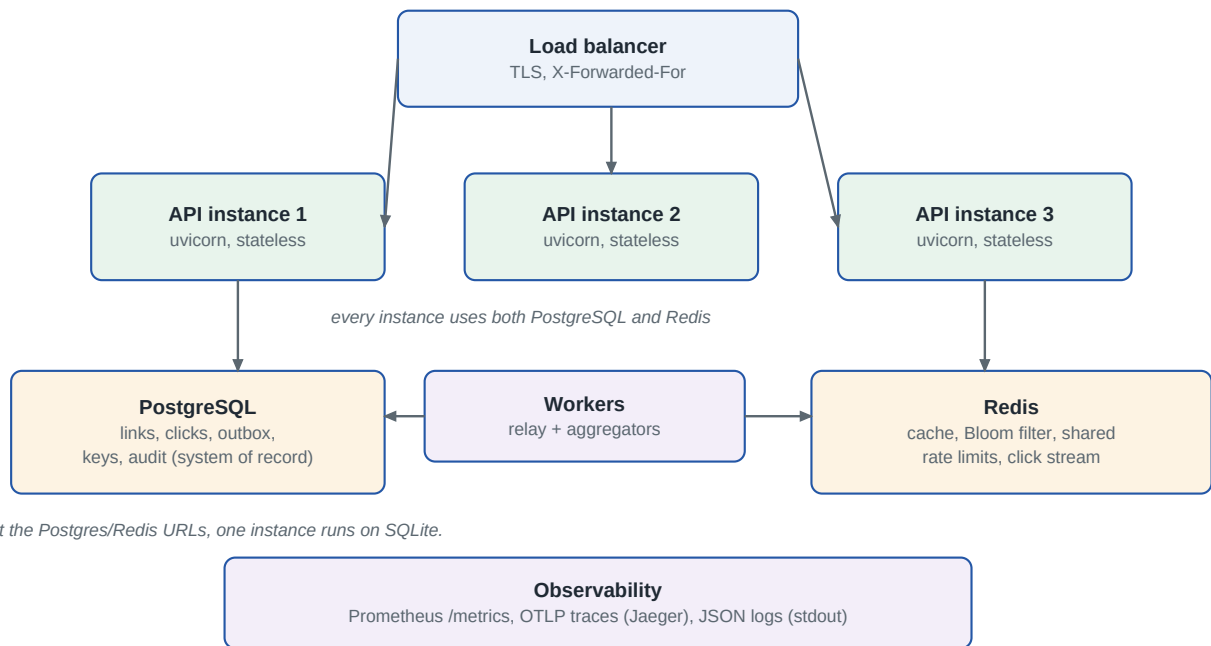


Figure 2. Scaled deployment; the optional parts are enabled by configuration.

Mode	Components	When to use
Single instance	One container or process; SQLite file; in-memory rate limits	Development, small deployments
Scaled	N stateless API instances; PostgreSQL; Redis (cache, Bloom filter, shared limits)	Production traffic, high availability
Events mode	Scaled mode plus relay and aggregator workers; Redis Streams	Decouple analytics writes; replay

Container: non-root user, health check, JSON logs on stdout, secrets from environment or mounted files.
Production mode refuses unsafe settings at startup.

7. Event pipeline (optional)

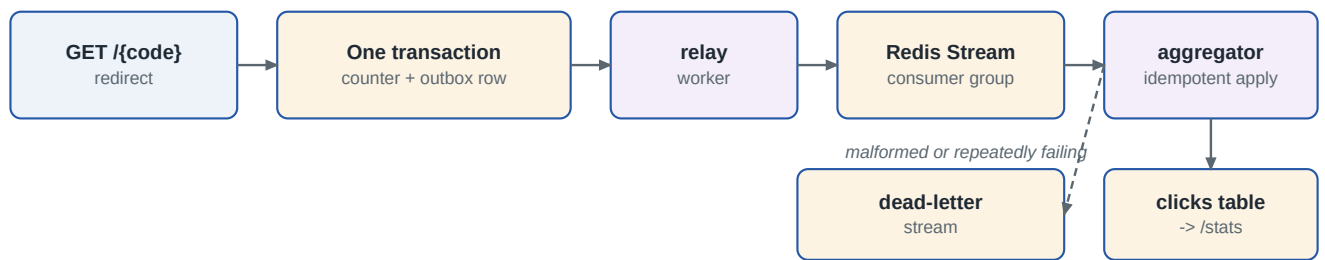


Figure 3. Click analytics in events mode.

Delivery is at-least-once; applying an event is idempotent (unique event_id), so duplicates and full replays never double-count. Counts and caps stay synchronous; only per-click details are asynchronous. The bus is a port (publish, read, ack, dead_letter); Redis Streams implements it, Kafka could.

8. Cross-cutting concerns

Concern	Mechanism
Security	API keys and roles, SSRF and look-alike checks, trusted proxies, security headers, secrets from files, production profile
Privacy	No raw IPs (daily keyed visitor IDs), referrer domains only, log redaction, private stats
Integrity	Hash-chained audit with per-caller identity; signed checkpoints for write-once storage
Observability	Prometheus metrics, OpenTelemetry traces, trace IDs in JSON logs, dashboards and alerts
Resilience	Circuit breaker and fail-fast 503 for Postgres; Redis failures fall back; throttled failure logs; readiness probe
Configuration	Defaults < TOML file < environment; validated at startup; secrets environment-only

9. Orchestrator architecture

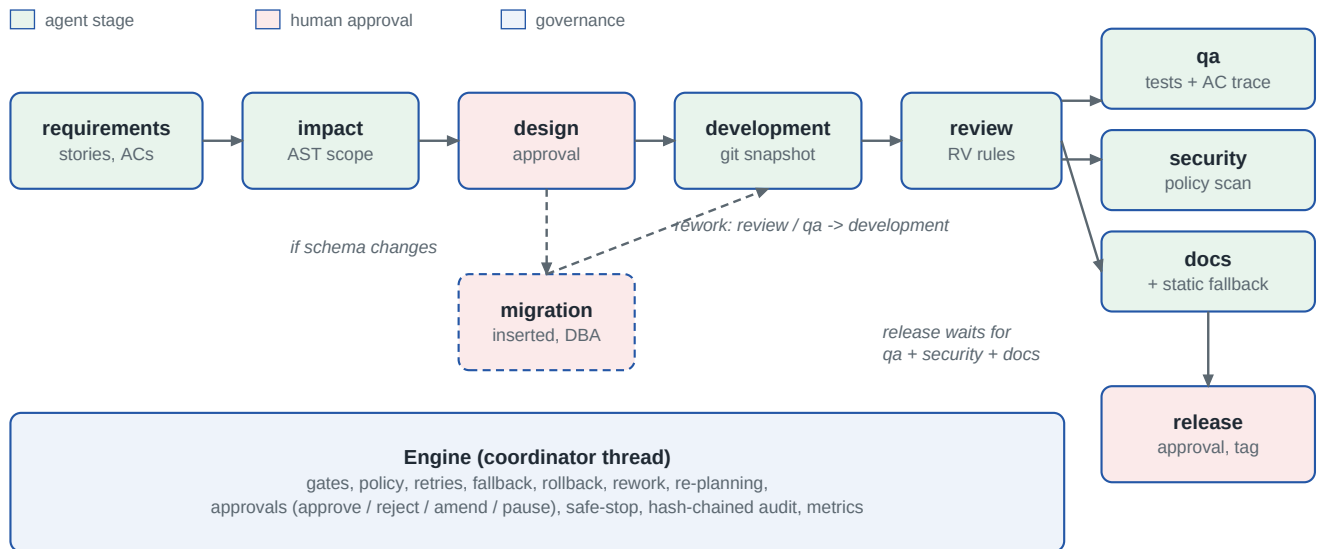


Figure 4. Stage graph used by the brownfield scenario; red boxes need human approval.

Element	Role
Engine	One coordinator thread owns all state transitions; agents run in parallel workers; per-stage generations discard stale results
Stage graph	Explicit dependencies, entry and exit gates; validated (no cycles); new stages can be inserted at run time
Policy engine	Security, compliance and change-control rules on every attempt; block (rollback and retry) or escalate (approval)
Approval gateway	Decision files or interactive; approve, reject, amend, pause; no default approval
Workspace	Git repository per run; snapshot before file changes; rollback on failure; commits promoted by a human
Context store	Versioned artifacts with lineage and decisions; persisted for pause and resume
Audit and metrics	Hash-chained run log; success rate, retries, rollbacks, reworks, MTTR, latency; run report
Agents	Requirements (+ LLM variant), impact, design, migration, development, review, QA, security, docs (+ fallback), release

10. Quality architecture

- CI: lint, strict typing, security scans, tests on Python 3.11 and 3.12 with separate 100% coverage gates, Docker integration with production-mode checks, infrastructure tests on real Postgres and Redis, and the SDLC scenarios.
- Evidence: load tests and chaos drills (docs/evidence), scenario run reports, generated traceability.